# YOUR VIBE-CODED APP IS GREAT.

# YOUR EMAILS ARE STUCK IN 2005.

// 01 — THE PROBLEM

# VIBE CODING IS GREAT. EXCEPT FOR EMAILS.

You build your app with Lovable, Claude, or Cursor. You ship the UI, auth, database, landing page, the works. Users sign up and they get an email. **And honestly, it's terrible...**

### WHY THIS HAPPENS

Because **email HTML is different from web HTML**. And AI doesn't know the constraints unless you explicitly tell it. So you get unbranded emails that break in Gmail, lose styles in Outlook, and go straight to spam.

- ✗ Generic email template
- ✗ Unstyled links
- ✗ Wrong font, wrong colours
- ✗ Dark mode chaos
- ✗ Broken spacing
- ✗ Landed in spam

// 02 — WHERE AI BREAKS EMAILS

# AI GENERATES WEB HTML. EMAIL NEEDS SOMETHING DIFFERENT.

## AI HANDLES THIS WELL

- ✓ Generating UI components
- ✓ Writing backend code
- ✓ Generating SQL
- ✓ Creating React components
- ✓ Writing copy
- ✓ Creating landing pages
- ✓ Fixing bugs

## BREAKS IN EMAILS

- ✗ Tailwind classes
- ✗ Flexbox layouts
- ✗ External CSS
- ✗ Custom fonts
- ✗ Modern HTML tags
- ✗ Dark mode styles
- ✗ Responsive layouts

// 03 — WHY DEFAULT AUTH EMAILS LOOK BAD

# DEFAULT TEMPLATES WORK. THEY JUST LOOK TERRIBLE.

**1**

Use default template

Safe but generic, no branding

**2**

Edit in dashboard

Manual edits, unsafe

**3**

Save

No preview, no version control

**4**

Break layout

HTML renders differently in clients

**5**

User gets broken emails

Brand damage, trust lost

THE RESULT

No branding. No versioning. No preview. No rollback. No safe deploy.
So people stick with the default — and **ugly emails ship forever**.

# SYSTEMIZE YOUR EMAILS.

FILE STRUCTURE

```
emails/
  layout.html      ← one base layout
  confirm.html     ← reuse structure
  invite.html
```

### One layout

A single base template. No duplication.

### Inline styles only

No external CSS. No Tailwind. Tables only.

### No runtime CSS or JS

Static, predictable, client-safe HTML.

### Small templates

One per email type. Easy to audit and test.

# GIVE CONSTRAINTS. GET GOOD OUTPUT.

✓ GOOD PROMPT

Generate an email-safe HTML template.

Use tables for layout.

Use inline styles only.

No flexbox. No external CSS.

Must work in Gmail and Outlook.

✗ BAD PROMPT

Make a nice email template

BE INTENTIONAL

AI works well for emails but only when you're specific.

// 06 — ALWAYS PREVIEW BEFORE SENDING

# NEVER DEPLOY EMAIL TEMPLATES BLIND.

1. **Preview in browser**
   Catch obvious layout breaks early
2. **Send test email**
   To a real inbox, not preview
3. **Check Gmail + Outlook**
   Both. Every time.
4. **Then deploy**
   With version history so you can rollback

### BROKEN WORKFLOW

Edit → Save → User receives → Panic

Most tools don't give you preview, version history, or rollback. So most builders skip all three to avoid user complaints.

### YOU NEED

- ✓ Preview
- ✓ Version history
- ✓ Rollback
- ✓ Safe deploy

# GOOD-LOOKING EMAILS THAT NEVER ARRIVE.

## MINIMUM SAFE SETUP

- ✓ Custom domain
- ✓ Verified sender address
- ✓ SPF record configured
- ✓ DKIM enabled
- ✓ Domain verified
- ✓ Tested in Gmail + Outlook

- ✗ SPF record missing
- ✗ DKIM not enabled
- ✗ Domain not verified
- ✗ Wrong sender address
- ✗ Using default SMTP
- ✗ Bad domain reputation

**MOST BUILDERS DO THIS LAST. DO IT FIRST.**

Set up domain → verify → SPF + DKIM → test → check spam → only then ship emails.

# SOLVING EMAIL PROBLEMS THAT EVERY VIBE-CODED APP RUNS INTO.

### Branded emails

Templates with preview-before-send and clean HTML export

### Event-triggered emails

Automated, not manual

### Template versioning

Never lose a working version again

### Deliverability diagnostics

Know before users complain

### Safe deploy & rollback

Ship with confidence

### Localisation

Emails that work for every market

# WANT EMAILS THAT VIBE WITH YOUR CUSTOMERS?

I'm building tools specifically for vibe-coded apps so your emails look as good as the rest of your product. Reach out below.

## HOW TO GET EARLY ACCESS

Message me on linkedin.com/in/chicocharlesworth for early access.